

RiscVFSM RISC-V Core

Eine schulische Erkläerung: welche Bausteine im Prozessor stecken, welche Aufgabe sie haben und wie aus vielen kleinen Schritten eine laufende CPU wird.

Stand: 05.05.2026

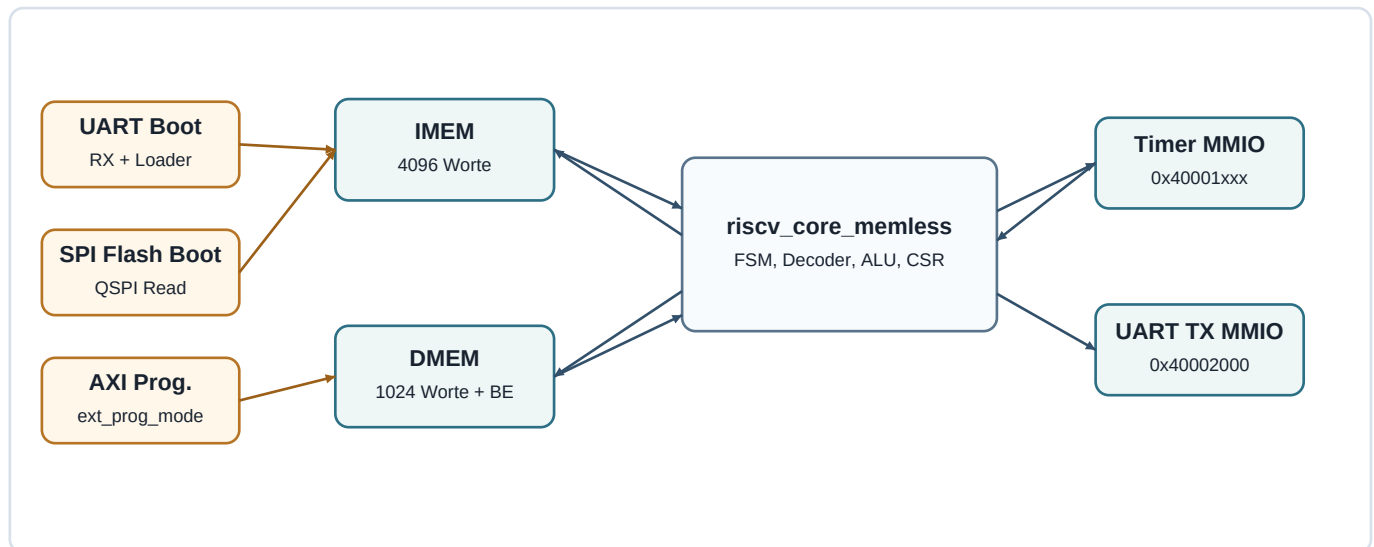
Quellenbasis: lokale VHDL-Dateien in `RiscVTest.srcs/sources_1/new`

Top-Pfad: `riscv_soc_boot` mit `riscv_core_memless`

1. Die Idee in einem Satz

Der Core ist ein kleiner RISC-V-Prozessor, der ein Programm Schritt fuer Schritt aus dem Instruktionsspeicher liest, die Befehle versteht, mit Registern und Rechenwerk ausfuehrt und bei Bedarf auf Speicher, Timer oder UART zugreift.

Er ist kein Pipeline-Prozessor. Stattdessen arbeitet er wie ein genauer Ablaufplan: holen, dekodieren, Operanden lesen, ausfuehren, eventuell Speicher benutzen und Ergebnis zurueckschreiben. Diese Ablaufsteuerung ist die FSM. Merksatz: Die FSM ist der Taktgeber des Denkens.



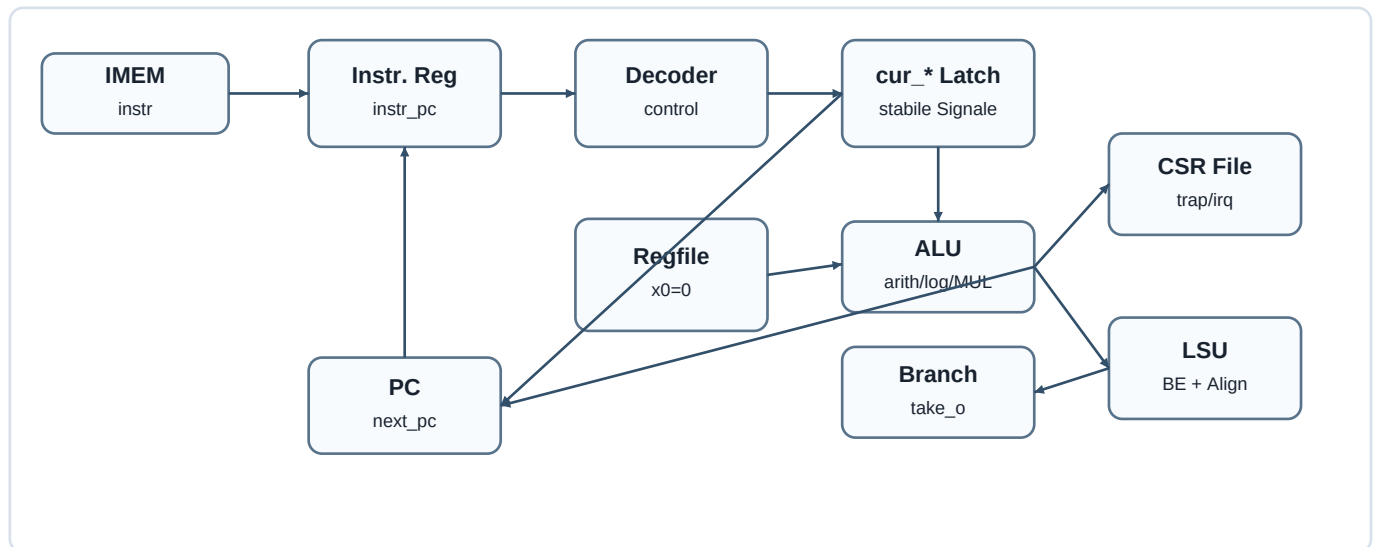
2. Was gehoert alles dazu?

Man kann den Aufbau wie ein kleines Computersystem sehen. Innen sitzt der eigentliche Prozessor. Aussen herum liegen Speicher, Boot-Logik und einfache Ein-/Ausgabe.

Baustein	Rolle im System	Zusammenhang
riscv_core_memless	Eigentlicher CPU-Kern mit PC, FSM, Decoder-Anbindung, Register, ALU, Branch, LSU und CSR.	Fragt Instruktionen aus IMEM ab und liest oder schreibt Daten ueber DMEM/MMIO.
decoder	Uebersetzer: erkennt aus den Instruktionsbits, ob es z. B. ADD, LW, Branch oder CSR ist.	Seine Ausgaenge werden im Core gespeichert, damit die naechsten FSM-Zustaende stabile Steuersignale haben.
regfile	Kurzzeitgedaechnis der CPU mit 32 Registern; x0 ist immer Null.	ALU, Branch Unit und LSU bekommen ihre Operanden meistens aus diesen Registern.
alu	Rechenwerk fuer Addition, Subtraktion, Bitlogik, Shifts, Vergleiche und MUL-Low32.	Ihr Ergebnis geht zurueck in ein Register oder dient als Speicheradresse.
branch_unit	Entscheider fuer bedingte Spruenge.	Bestimmt, ob der PC normal weiterlaeuft oder zum Branch-Ziel springt.
load_store_unit	Adapter zwischen CPU-Werten und byteweisem Speicherzugriff.	Setzt Byte-Enables und erweitert Load-Ergebnisse passend auf 32 Bit.
csr_file	Verwaltungsbereich fuer Traps, Interrupts und Watchdog.	Liefert bei besonderen Ereignissen die naechste PC-Adresse und merkt Ursache/Zustand.
riscv_soc_boot	SoC-Schale um den Core.	Verbindet Bootloader, Speicher, Timer, UART, Reset und CPU-Core.

3. Der Weg eines Befehls

Ein RISC-V-Befehl ist fuer die Hardware zuerst nur ein 32-Bit-Muster. Damit daraus eine Aktion wird, wandert dieses Muster durch mehrere Stationen: Fetch, Decode, Register lesen, Execute, eventuell Memory und Writeback.



Nicht jeder Befehl benutzt alle Stationen gleich. Ein ADD braucht keinen Datenspeicher. Ein SW schreibt in den Speicher und hat keinen Register-Writeback. Ein Branch entscheidet nur, welcher PC als naechstes gilt.

4. Die FSM als Ablaufplan

Die Zustandsmaschine bringt Ordnung in den Ablauf. Sie sorgt dafuer, dass erst eine Instruktion geholt, dann verstanden, danach mit Registerwerten ausgefuehrt und am Ende sauber abgeschlossen wird.

Phase	Schulische Erklaerung	Typische Zustaeude
Start und Holen	Die CPU nimmt den PC und fragt damit den Instruktionsspeicher.	ST_RESET, ST_FETCH, ST_FETCH_WAIT
Verstehen	Der Decoder zerlegt die Instruktion in Registeradressen, Immediate und Steuersignale.	ST_DECODE
Vorbereiten	Das Registerfile liefert die benoetigten Registerwerte.	ST_RF_WAIT
Ausfuehren	ALU, Branch Unit, LSU oder CSR-Logik erledigen die eigentliche Aufgabe.	ST_EXEC_ALU, ST_EXEC_BRANCH, ST_EXEC_LOAD_ADDR, ST_EXEC_STORE_ADDR, ST_EXEC_CSR
Abschliessen	Register, Speicher oder PC werden aktualisiert.	ST_MEM_READ, ST_MEM_WRITE, ST_WB, ST_TRAP, ST_MRET

Weil der Core mehrzyklisch ist, kann man ihn gut beobachten: `state_dbg_o` zeigt, in welchem Schritt die CPU gerade steht.

5. Decoder, Registerfile und ALU

Diese drei Bausteine bilden den normalen Rechenweg. Der Decoder sagt, welche Register gelesen werden und welche Operation gebraucht wird. Das Registerfile liefert die Zahlen. Die ALU fuehrt die Operation aus. Danach schreibt der Core das Ergebnis in das Zielregister zurueck.

Beispiel ADD x3, x1, x2: Der Decoder erkennt: lies x1 und x2, addiere beide Werte und schreibe das Ergebnis nach x3. Das Registerfile liefert die Eingaben. Die ALU addiert. Im Writeback-Zustand bekommt x3 das Ergebnis.

Beispiel ADDI x3, x1, 5: Der zweite Wert kommt nicht aus einem Register, sondern direkt aus der Instruktion. Der Decoder erzeugt den Immediate-Wert 5. Die ALU addiert x1 und 5.

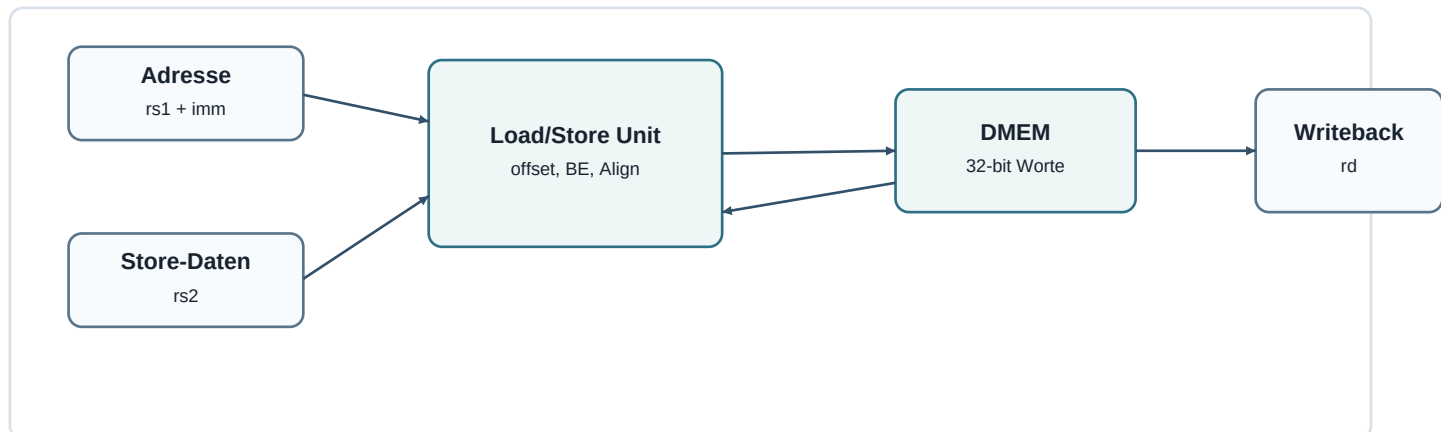
6. Branches

Normalerweise wird nach jedem Befehl der PC um 4 erhoeht. Ein Branch kann diesen normalen Weg aendern. Die Branch Unit vergleicht zwei Registerwerte und entscheidet, ob das Branch-Ziel oder PC+4 der naechste PC wird.

Zusammenhang: Der Decoder erkennt die Branch-Art, das Registerfile liefert die Vergleichswerte, die Branch Unit entscheidet, und die FSM schreibt danach den passenden PC.

7. Speicherzugriffe und LSU

Die CPU rechnet intern mit 32-Bit-Werten. Der Speicher kann aber auch einzelne Bytes oder Halfwords lesen und schreiben. Die Load/Store Unit uebersetzt deshalb zwischen CPU-Wunsch und 32-Bit-RAM mit vier Byte-Lanes.



Bei einem Store erzeugt die LSU die passenden Byte-Enables. Bei einem Load sucht sie die richtige Byte-Lane heraus und erweitert den Wert auf 32 Bit. LB/LH erweitern mit Vorzeichen, LBU/LHU mit Nullen.

8. CSR, Traps, Timer und Watchdog

Nicht jeder PC-Wechsel kommt aus einem normalen Branch. Bei illegalen Befehlen, ECALL, Timer-Interrupt oder Ruecksprung aus einer Ausnahmebehandlung wird die CSR-Datei wichtig.

CSRs merken, wo die CPU vor einer Ausnahme war (`mepc`), warum die Ausnahme passiert ist (`mcause`) und wohin gesprungen werden soll (`mtvec`). Der Watchdog kann einen Reset erzeugen, wenn er nicht rechtzeitig neu geladen wird.

9. Die SoC-Schale

Ein CPU-Core allein reicht noch nicht, denn irgendwoher muss das Programm kommen und irgendwohin muessen Daten geschrieben werden. `riscv_soc_boot` verbindet den Core mit Bootloadern, Speicher, Timer, UART und Reset-Logik.

Bootpfad	Einfache Erkl��rung
UART	Ein Programm kann byteweise ueber die serielle Schnittstelle geladen werden.
SPI	Ein Programm kann aus einem externen Flash gelesen werden.
External/AXI	Eine aeussere Logik kann IMEM/DMEM direkt programmieren und den Start steuern.

MMIO bedeutet: Peripherie wird wie Speicher angesprochen. Ein Schreibzugriff auf `0x40002000` sendet zum Beispiel ein UART-Byte. Der Timer liegt in der Region `0x40001000` bis `0x40001FFF`.

10. Gesamtbild

Der PC zeigt auf den naechsten Befehl. IMEM liefert diesen Befehl. Der Decoder macht daraus Steuersignale. Das Registerfile liefert Werte. ALU, Branch Unit, LSU oder CSR-Logik erledigen die Aufgabe. Die FSM fuehrt alles in der passenden Reihenfolge aus.

Dadurch ist der RiscVFSM-Core fuer ein Lehr- und Bring-up-Projekt gut geeignet: Die Bausteine sind getrennt genug, um sie einzeln zu verstehen, aber eng genug verbunden, um zu sehen, wie daraus ein echter Prozessor wird.

Hinweis: Diese Dokumentation beschreibt den lokalen Stand der Dateien unter `RiscVTest.srcs/sources_1/new`. Generierte Vivado/IP-Shared-Kopien wurden nicht als primaere Quelle verwendet.